# Reverse image search using centroids of vector embeddings

Reverse image search (RIS) of a single image is not always descriptive enough to find every image in a class. Leveraging RIS by using multiple images of that class can improve results – however, exhaustively searching a database for all related images in this way is computationally intensive. Searching with an aggregate representation of all the images would be more focused and inexpensive. We test whether a centroid of vector embeddings is an effective means of searching for a class of images.

I. MOTIVE

We want to use Reverse Image Search (RIS) to search for a class of images. A class of images is a set of images which have something in common. We already have some images of that class and we want to find more from a database.

As an example, say we have a picture of a cat, and we want to search a database to find every cat picture. We can use RIS methods to try to find the other cat pictures from the database. But there are some problems with this approach. One problem is that searching with only one cat picture is not representative of the whole class of cats, an example of this would be searching with a picture which depicts a black and white tabby cat in a field and not finding a picture which depicts a close up brown tabby cat's face. Since they are different enough, these images may not find each other through RIS.

FIG. 1. Comparison of cat pictures with major differences.

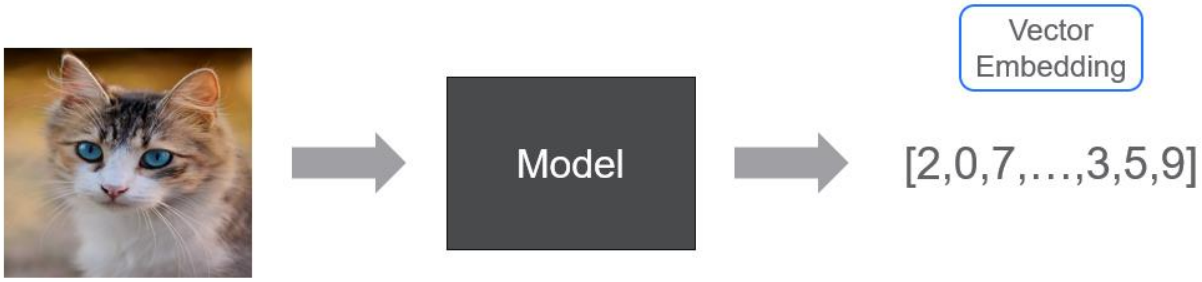


To mitigate this, we need to search using multiple pictures, that way the class of images we are searching for can be better represented. But searching with multiple images requires searching multiple times. Searching that much can be slow, especially with large databases. We need a way to represent the whole class of images without taking so much time search

## II. BACKGROUND

There are many ways to implement RIS. In our case, we used a machine learning model which classifies images into a broad range of classes[1]. The model uses the Xception architecture[2] and was trained on ImageNet[3]. We use a modified version of the model which produces embedding vectors, rather than classifications[4]. To use it, we give the model input images, and the model produces Vector embeddings of those images.

FIG. 2. Process of vectorizing




Vectors are a list of numbers. [1,5,2,4] would be an example of a vector with 4 dimensions. Vector embeddings are vectors which hold meaning about the original data. In our case, the vector embeddings hold meaning about the input image. Our vector embeddings have 2048 dimensions (2048 different numbers in them). The information that the vector embeddings have is based on how the model was trained.

These vector embeddings can be used to search. If we vectorize every image in our database and store the produced vector with the image, we can compare the vector embeddings and return images which have similar vectors. Vectors can be compared by using a distance equation of some kind.

Vectors with many dimensions cannot be visualized geometrically. So we give examples here of 2 dimensional vectors. The processes are the same for higher dimensions, but are much easier to understand with 2 dimensional vectors.

---

[1] IMAGENET 1000 Class List, https://deeplearning.cms.waikato.ac.nz/user-guide/class-maps/IMAGENET/

[2] **Xception: Deep Learning with Depthwise Separable Convolutions,** F. Chollet, arXiv 2017, DOI: 10.48550/arXiv.1610.02357, https://doi.org/10.48550/arXiv.1610.02357

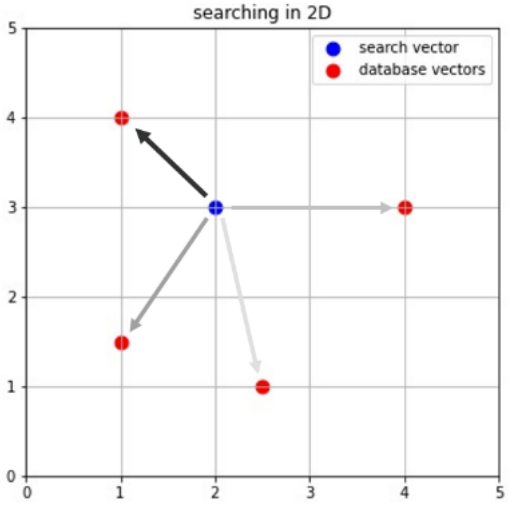
[3] J. Deng et al., "ImageNet: A large-scale hierarchical image database," 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248-255, doi: 10.1109/CVPR.2009.5206848.

[4] xception.py, F. Chollet et al., Published by: Github, https://github.com/keras-team/keras/blob/v2.10.0/keras/applications/xception.py#L84

Here is a visualization of a search using 2-dimensional vectors

The blue vector represents the vector of the image we searching with. Red vectors represent the vectors of images in the database. The search algorithm gives results based on which database vectors are closest to the search vector. The database image with the closest vector is returned first, then the next closest, and so on.

FIG. 3. Searching in 2 dimensions using Euclidean distance

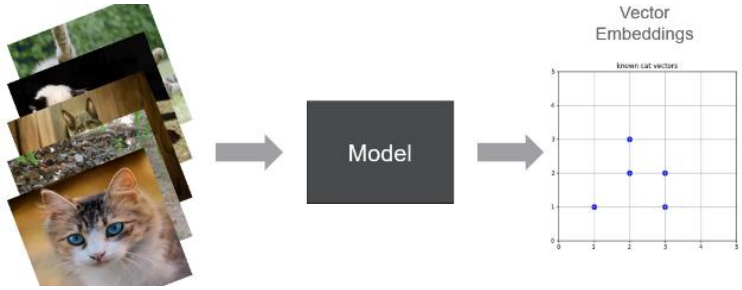


A. Centroid

Revisiting the problem, we want a search method which is both representative of a class, and searches quickly. We also have vectors representing images. So, instead of representing multiple images, we can try to represent multiple vectors. To do this, we decided to make a centroid out of the vectors made from the search images. A centroid is like an average of vectors.

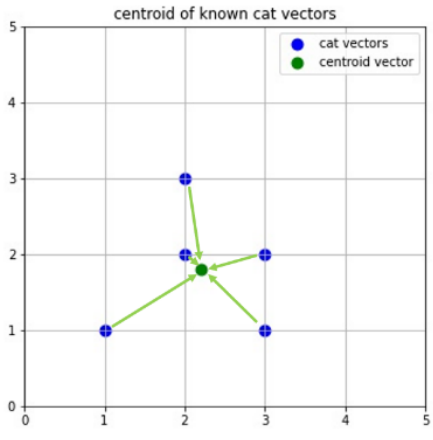
To make a centroid, first we vectorize the images that we want to search with.

FIG. 4. Vectorizing many pictures of a cat class



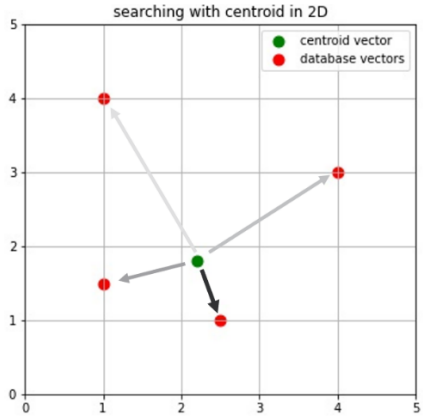

Next, we take the produced vectors and average their values on each axis. This creates the centroid.

FIG. 5. Creating a centroid from vectors in 2 dimensions



Afterward, we can use the centroid to search the database.

FIG. 6. Searching with a centroid in 2 dimensions using Euclidean distance




## III. TEST METHOD

To test whether a centroid is an effective solution, we had to create a method of testing both the effectiveness of the centroid, and the baseline effectiveness of just using single images

### A. Setup

To setup our experiments, we looked online to find a dataset of images labelled by class[5,6]. Then, for each class of images, we divided the images in that class into two subsets, a target set and a search set. The images from the target set were put into the database. The images in the search set were kept out of the database. During experiments we used the search set images to query the database for the target set images of the same class.

[5] J. Deng et al., "ImageNet: A large-scale hierarchical image database," 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248-255, doi: 10.1109/CVPR.2009.5206848.
[6] Natural Adversarial Examples, K. Z. Dan Hendrycks et al., arXiv 2021, DOI: arXiv:1907.07174, https://arxiv.org/abs/1907.07174

B. Testing

After the database and search images were set up, we ran two sets of experiments.

1. *Baseline test*

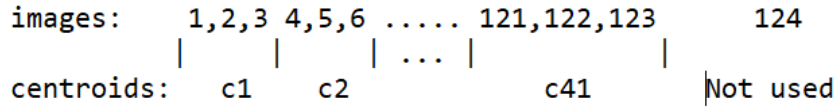
We needed to test the efficacy of searching without using a centroid. To do this we ran queries using single images from the search set to find the target images of the same class. We did this for every class. There were 124 search set images per class, so we did 124 queries per class.

2. *Centroid test*

Next, we ran an experiment to test the efficacy of searching with a centroid. To do this, we took the same search images, and made centroids out of them, then searched with those centroids. Centroids can be made out of different numbers of images. And we supposed that a centroid which used more images may give better results than a centroid which used fewer images. So we tested centroids which used different numbers of images. Each time, we split the search set into groups of N images, each group of N images would then be used to build a centroid. Then we used that centroid to query. We tested a few different Ns: 3,5,10,20,40, and 80. Each time, we used the same pool of 124 search images and had no overlap in the images used between centroids. So, as we used more images in the centroids, the number of queries we could do for each experiment was fewer. So there are less datapoints in the histograms with large N values. See appendix for how many queries there were per N value.

To illustrate using N images, lets look at an example of creating centroids from groups of 3 images. Each centroid is created by choosing a group of 3 images, then the next centroid is made using the next 3 images, etc. if there were less than 3 images, then those images were ignored.

FIG. 7 Choosing groups of images to use in each centroid




1,2,3… = images

c1,c2,c3…. = centroids

## IV. IMPLEMENTATION

### A. Queries

The database application we used to perform searches is OpenSearch[7]. OpenSearch has its own *distance score* functions, which are modifications of commonly used distances: City-block distance (L1), Euclidean distance (L2), L-infinity, Inner product, and Cosine similarity. [see appendix for equations]. Queries compared the search vector to the database vectors and returned vectors with the highest distance scores, instead of smallest distance. We used distance score functions rather than distances because that is how OpenSearch implements its search methods[8].

### B. Dataset

The dataset we used for the experiments was made of three datasets put together. Tiny ImageNet[9], ImageNet Natural-Adversarial Examples[10] (ImageNet-A), and ImageNet Out-of-Distribution Examples[11] (ImageNet-O). Tiny ImageNet is a subset of the original ImageNet[12] set. ImageNet-A is made of images which trick image classification models that were trained on ImageNet into classifying the images as the wrong class. So, they trick the model into giving false negatives. ImageNet-O is made of images which are not part of the classes in ImageNet, but mimic classes in ImageNet. So, they trick the model into giving false positives. Overall, there were 20 image classes shared between the 3 sets. We only used those shared classes.

The reason we chose these datasets is because we thought the model would be able to vectorize the images with some sort of predictability while still having images which are difficult to classify. If we had instead used random images found off the internet, then there would be no intuition for how the model would behave. The downside of using this dataset is that we do not know the effectiveness of this technique for images outside of the original ImageNet class set. But that would be for a future experiment.

For our experiments, we split the images for each class of the dataset into a target set and a search set. The target set for each class consisted of 300 images from Tiny ImageNet, an average of 32 images from ImageNet-A, and an average of 10 images from ImageNet-O.

For the search set, each class had 124 images from only Tiny ImageNet.

---

[7] https://opensearch.org/

[8] https://opensearch.org/docs/latest/search-plugins/knn/knn-score-script/#spaces

[9] https://github.com/rmccorm4/Tiny-Imagenet-200

[10] Natural Adversarial Examples, K. Z. Dan Hendrycks et al., arXiv 2021,DOI: arXiv:1907.07174, https://arxiv.org/abs/1907.07174

[11] Natural Adversarial Examples, K. Z. Dan Hendrycks et al., arXiv 2021,DOI: arXiv:1907.07174, https://arxiv.org/abs/1907.07174

[12] J. Deng et al., "ImageNet: A large-scale hierarchical image database," 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248-255, doi: 10.1109/CVPR.2009.5206848.

# VI. RESULTS

## A. Measurement metric

To measure how good our results were we used a metric called R-Recall.

R-Recall[13] is based off of Recall. Recall is calculated by dividing the number of found images in the target set, by the number of images in the target set. It is the percent of target images found.

So if we found 4 cats, and there were 10 cats in the database, then that query had a recall of 40%.

$$\text{recall} = \frac{|\{\text{relevant documents}\} \cap \{\text{retrieved documents}\}|}{|\{\text{relevant documents}\}|}$$ [14]

Recall has a problem though, if every image is returned in the database, then recall will always be 100% because all target images would be found. R-Recall tries to fix this problem. R Recall is the same as recall, but we only look at the first R images returned by a query, where R is the number of target images we want to find. Our target sets have about 342 images each, so we only check the closest 342 images of our query results, and calculate recall using those.

[13] Evaluation measures (information retrieval), Published by: Wikipedia Last Updated: 29 November 2022, https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)#R-precision

[14] Evaluation measures (information retrieval), Published by: Wikipedia Last Updated: 29 November 2022, https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)#Recall

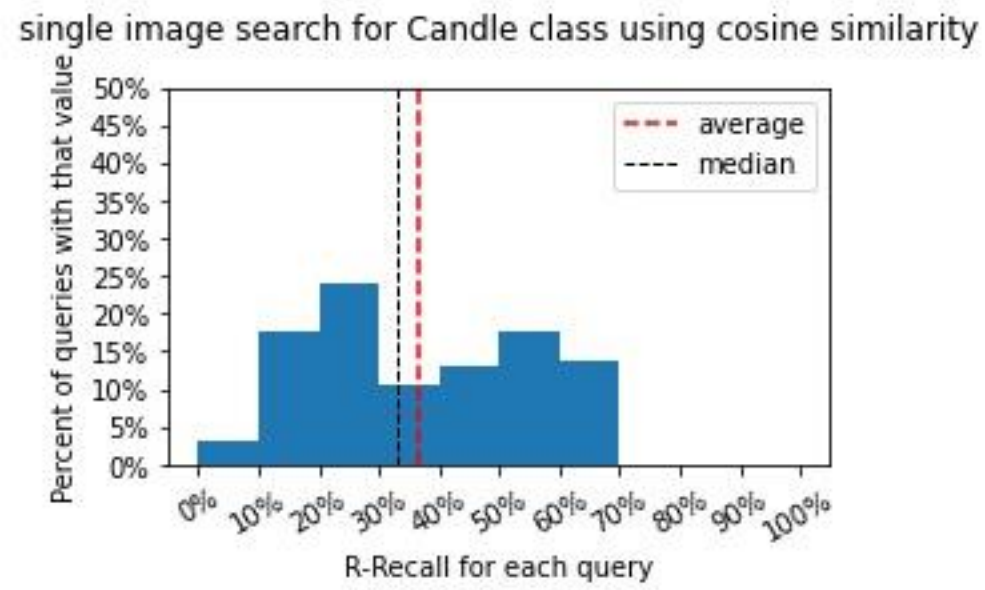
B. Baseline experiment

We did the baseline experiment using a variety of distance scores, but the main one we will be focusing on is cosine similarity because that had the best results for both single searches and centroid searches. All results from other search scores are shown in the appendix.

Below are histograms of the R-Recall values for all the queries which used single images. The X axis shows R-Recall values and is divided into bins with widths of 10%. The Y axis shows what percentage of queries had an R-Recall value which was in that bin. So in this first graph, very few queries had an R-Recall between 0%-10% but many queries had an R-Recall between 20%-30%.

First, here are the results for one class of images

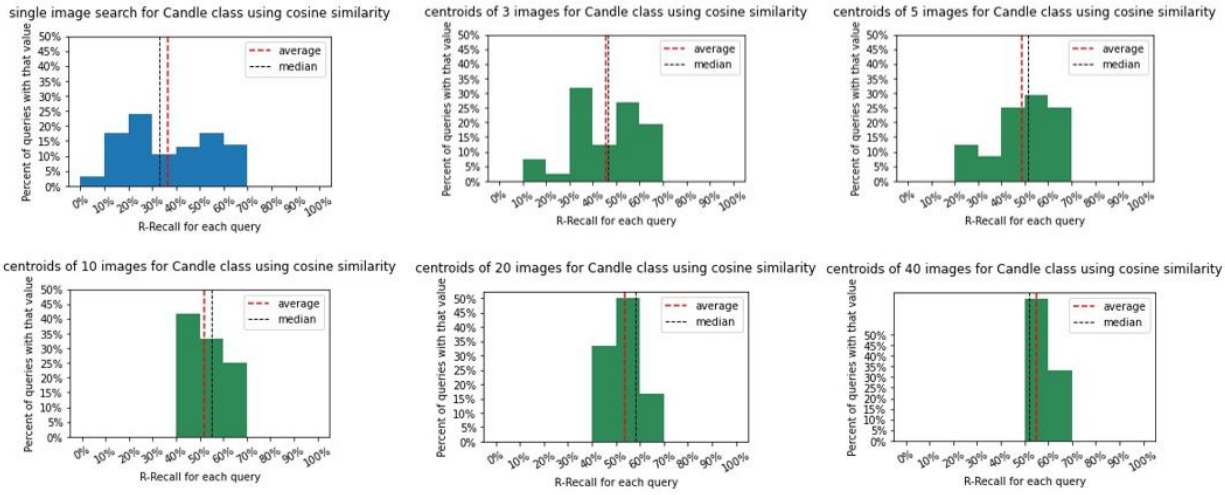
FIG. 8. Results of queries using single images, for Candle class images, using cosine similarity scores.




C. Comparison with Centroid

FIG. 9 Results of queries using centroids made using different numbers of images, for Candle class images, using Cosine Similarity scores.

Top Left is Figure 8 for comparison. Top middle shows the results of using centroids made from 3 images. Top right uses 5 images per centroid, The bottom left, middle, and right graphs show the results of using 10, 20, and 40 images per centroid, respectively.

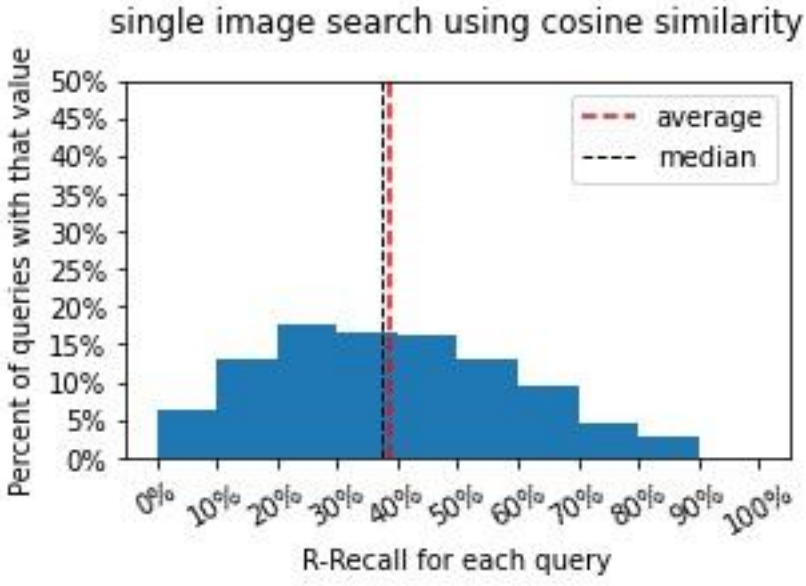



We can compare how the distribution of R-Recall changes as we use the same images in centroids. The median and mean shift to the right as more images are used in the centroid. This means that the average results are improving with more images. The distribution's minimum also increases, which also shows improvement.

D. Results from all classes

Next, here are the results compiled from all 20 classes.

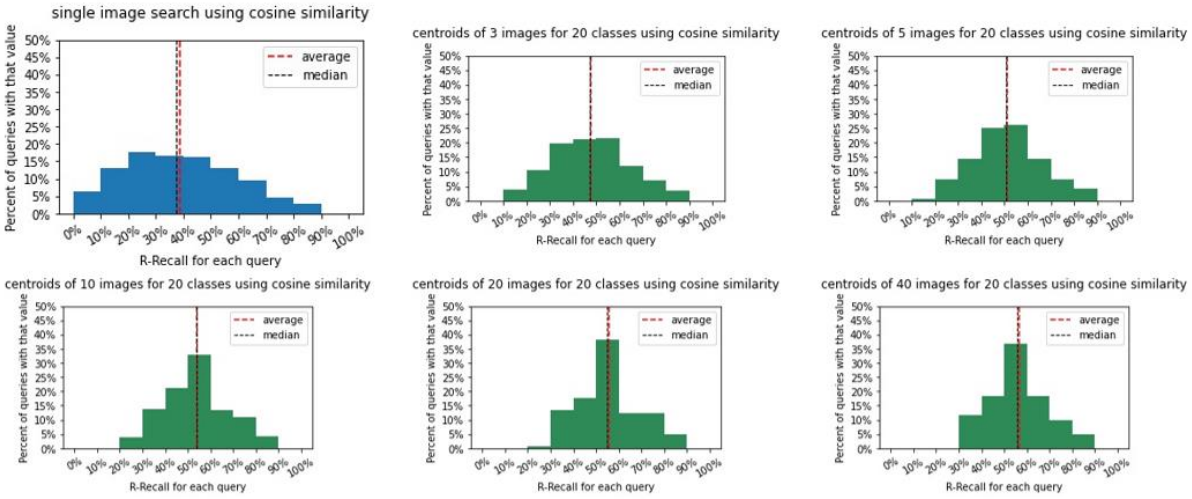
FIG. 10. Results of queries using single images, aggregated across 20 classes, using Cosine Similarity scores.




E. Comparison with Centroid

FIG. 11. Results of queries using centroids made using different numbers of images, aggregated across 20 classes, using Cosine Similarity scores.

Top Left is Figure 10 for comparison. Top middle shows the results of using centroids made from 3 images. Top right uses 5 images per centroid, The bottom left, middle, and right graphs show the results of using 10, 20, and 40 images per centroid, respectively.




28

The distribution of R-Recall changes in the same way as the distribution from a single class does.

## VII. CONCLUSIONS

We can see that as more images are used in the centroid, the distribution of R-Recall results changes. The average and median increase. the minimum increases as well. And the overall distribution tends to be closer to the centroid. This means that as more images get used in the centroid, the results tend to improve, and they also become more consistent.

Overall, using a centroid improves search results over using a single image. Using more images to make the centroid will improve the search results further, but after somewhere between 10-20 images, the improvements are small. Centroids also have results which are less varied, and closer to their improved average.

## VIII. FURTHER WORK

There are other techniques we can try which may give better results than using a centroid. Using techniques based on K-means or hierarchical clustering seem promising.

It would be interesting to look at how the characteristics of the images and vectors which go into a centroid affect how well that centroid does.

We could make the results more robust by running the same experiment but on datasets which have no overlap with the ImageNet classes.

## IX. IMPACT

Having better, faster search results is very helpful. This work shows improvements that can be made to AI-based reverse image search without modifying the original model. This is important because model retraining is expensive, so having solutions which do not require retraining are more cost-effective. Having inexpensive solutions makes it possible for people to better do what they need to do, without extra work.

## X. ACKNOWLEDGEMENTS


- Derek Tropf (Mentor)
- Melissa Swift
- Joshua Harrison
- Amanda White
- Tegan Emerson
- This work was supported in part by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists (WDTS) under the Science Undergraduate Laboratory Internships Program (SULI).

# XI. APPENDIX

## APPENDIX: NUMBER OF QUERIES PER NUMBER OF IMAGES USED IN A CENTROID

| Number of images used to make a centroid | Number of queries per class |
|---|---|
| 1 | 124 |
| 3 | 41 |
| 5 | 24 |
| 10 | 12 |
| 20 | 6 |
| 40 | 3 |
| 80 | 1 |

Number of queries per class = floor(124/N) where N is the number of images used to make a centroid.

## APPENDIX: DISTANCE SCORE EQUATIONS[15]

City-Block score (L1)

$$score(x,y) = \frac{1}{1 + sum(|xi - yi| \; for \; all \; xi, yi \; in \; x, y)} \quad (1)$$

Euclidean score (L2)

$$score(x,y) = \frac{1}{1 + sum((xi - yi)^2 \; for \; all \; xi, yi \; in \; x, y)} \quad (2)$$

Supremum score (L infinity)

$$score(x,y) = \frac{1}{1 + max(|xi - yi| \; for \; all \; xi, yi \; in \; x, y)} \quad (3)$$

Inner product score

$$d = -x * y$$
$$score(x,y) = if \; d \geq 0: \frac{1}{1+d} \; else: \; -d + 1 \quad (4)$$

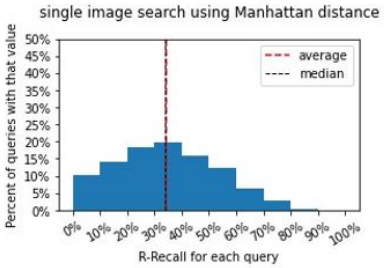
Cosine Similarity score

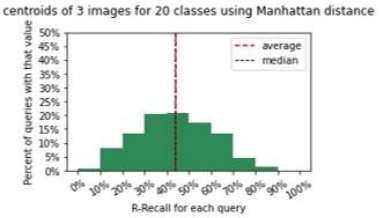
$$score(x,y) = 1 + \frac{x * y}{||x|| * ||y||} \quad (5)$$

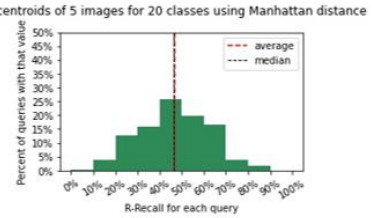
[15] https://opensearch.org/docs/latest/search-plugins/knn/knn-score-script/#spaces

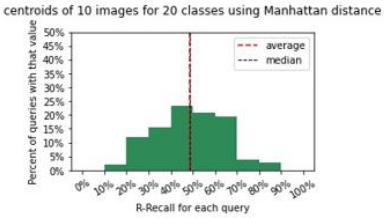
# APPENDIX: R-RECALL IN OTHER SPACES FOR ALL 20 CLASSES

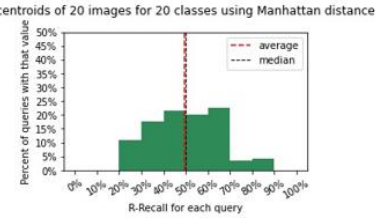
## Manhattan Distance (L1) score

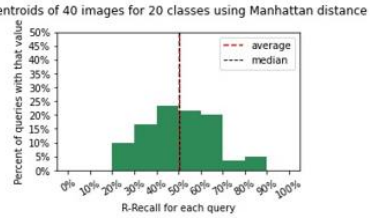


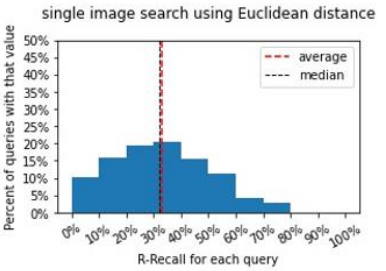



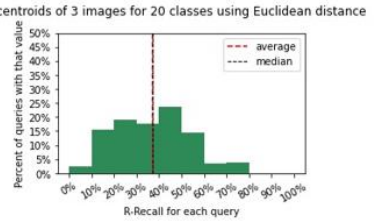



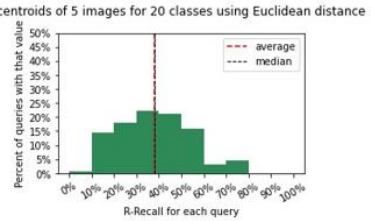



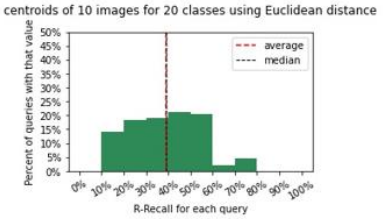



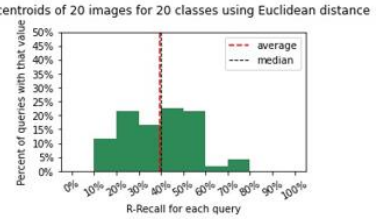

## Euclidean Distance (L2) score

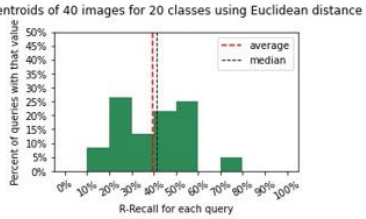


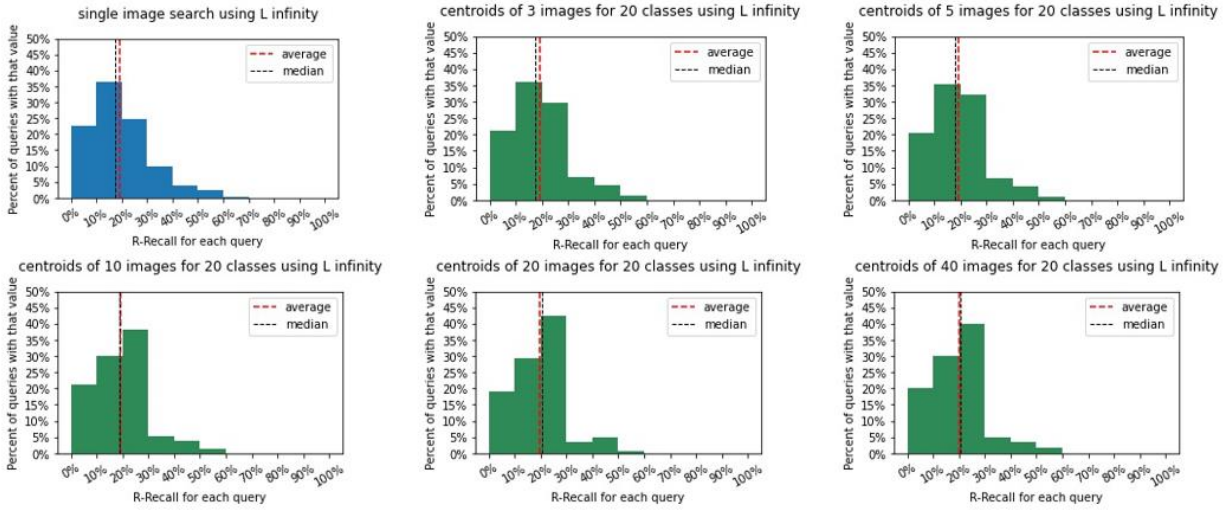



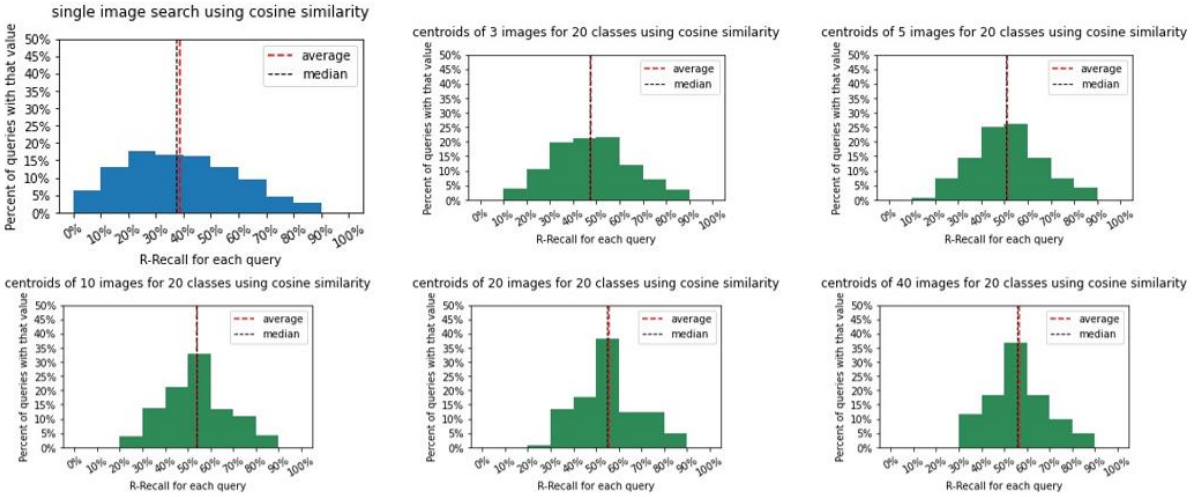



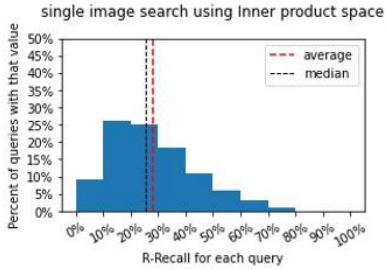
## L-infinity score

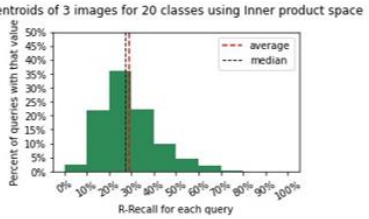


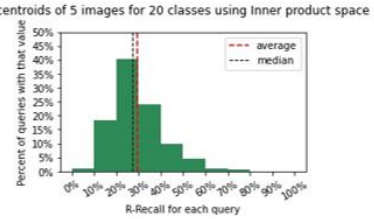

## Cosine-Similarity score

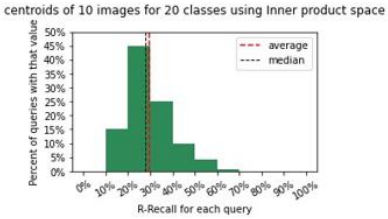


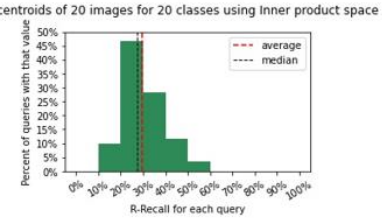
Inner-product score

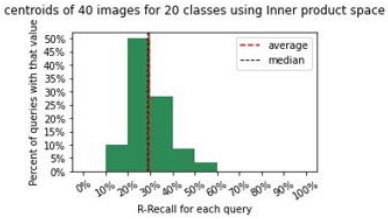


# REFERENCES

J. Deng, W. Dong, R. Socher, L. -J. Li, Kai Li and Li Fei-Fei, "ImageNet: A large-scale hierarchical image database," *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248-255, doi: 10.1109/CVPR.2009.5206848.

Evaluation measures (information retrieval), Published by: Wikipedia Last Updated: 29 November 2022, https://en.wikipedia.org/wiki/Evaluation_measures_(information_retrieval)#Recall

IMAGENET 1000 Class List, https://deeplearning.cms.waikato.ac.nz/user-guide/class-maps/IMAGENET/

Nearest Neighbor Indexes for Similarity Search, J. Briggs, https://www.pinecone.io/learn/vector-indexes/

Xception: Deep Learning with Depthwise Separable Convolutions, F. Chollet, arXiv 2017, DOI: 10.48550/arXiv.1610.02357, https://doi.org/10.48550/arXiv.1610.02357

Natural Adversarial Examples, K. Z. Dan Hendrycks, Steven Basart, Jacob Steinhardt, Dawn Song, arXiv 2021, DOI: arXiv:1907.07174, https://arxiv.org/abs/1907.07174

xception.py, Published by: Github, https://github.com/keras-team/keras/blob/v2.10.0/keras/applications/xception.py#L84

https://opensearch.org/

https://opensearch.org/docs/latest/search-plugins/knn/knn-score-script/#spaces

https://github.com/rmccorm4/Tiny-Imagenet-200